

Formal Languages and Compilers

Prof. Breveglieri, Morzenti, Agosta

Written exam: laboratory question

13/06/2024

Time: 60 minutes. Textbooks and notes can be used. Pencil writing is allowed.

Important: Write your name on any additional sheet.

SURNAME (Cognome):

NAME (Nome):

Matricola:or Person Code:

Instructor: ☐ Prof. Breviglieri ☐ Prof. Morzenti ☐ Prof. Agosta

The laboratory question must be answered taking into account the implementation of the Acse compiler given with the exam text.

Modify the specification of the lexical analyser (`flex` input) and the syntactic analyser (`bison` input) and any other source file required to extend the Lance language with the new **if-repeat-until** statement. This new statement has the following syntax:

if_repeat (*<exp. 1>*) *<block>* **until** (*<exp. 2>*);

The `if-repeat-until` statement is a loop controlled by the two expressions *<exp. 1>* and *<exp. 2>*. *<exp. 1>* is only evaluated before the loop itself: if it is false (i.e. equal to zero), the loop is not executed. Instead, when *<exp. 1>* is true (i.e. different than zero) the *<block>* is executed in a loop which *stops* when *<exp. 2>* is evaluated to true. In all cases *<exp. 1>* is only evaluated once.

The following program exemplifies the operation of the `if-repeat-until` statement. First, an integer value is read from standard input and assigned to variable *i*. Then, the `if-repeat-until` statement checks if *i* < 3. If that is not the case, the entire statement is skipped and execution continues at the “`write(99);`” line. For instance, if the number inserted is 5, the program skips the loop and only prints 99.

If the *i* < 3 condition is met, the loop starts executing. At each iteration, the loop body prints *i* and then increments it. The loop stops when *i* reaches 6, and then the program reaches the “`write(99);`” line. For instance, if the number inserted is 2, the program prints 2 3 4 5 99. Note that – while performing the loop – *i* can violate the *i* < 3 condition, as it is only checked before the loop.

```
int i;
read(i);
if_repeat (i < 3) {
    write(i);
    i = i + 1;
} until (i == 6);
write(99);
```

1. Define the tokens (and the related declarations in **Acse.lex** and **Acse.y**). (3 points)
2. Define the syntactic rules or the modifications required to the existing ones. (4 points)
3. Define the semantic actions needed to implement the required functionality. (18 points)

Solution

The solution is shown below in *diff* format. All lines that begin with '+' were added, while the lines that begin with '-' were removed. **It is not required and it is not encouraged to provide a solution in *diff* format to get the maximum grade.**

```
diff --git a/acse/Acse.lex b/acse/Acse.lex
index 35f9b73..665c0e7 100644
--- a/acse/Acse.lex
+++ b/acse/Acse.lex
@@ -93,6 +93,8 @@ ID      [a-zA-Z_][a-zA-Z0-9_]*
"return"      { return RETURN; }
"read"        { return READ; }
"write"       { return WRITE; }
+ "if_repeat"  { return IF_REPEAT; }
+ "until"      { return UNTIL; }

{ID}          { yylval.svalue=strdup(yytext); return IDENTIFIER; }
{DIGIT}+      { yylval.intval = atoi( yytext ); }
diff --git a/acse/Acse.y b/acse/Acse.y
index 5e31c71..8dc5971 100644
--- a/acse/Acse.y
+++ b/acse/Acse.y
@@ -109,6 +109,7 @@ extern void yyerror(const char*);
    t_list *list;
    t_axe_label *label;
    t_while_statement while_stmt;
+   t_if_repeat_stmt if_repeat_stmt;
}
/*=====
                                TOKENS
@@ -125,6 +126,7 @@ extern void yyerror(const char*);
%token RETURN
%token READ
%token WRITE
+ %token UNTIL

%token <label> DO
%token <while_stmt> WHILE
@@ -133,6 +135,7 @@ extern void yyerror(const char*);
%token <intval> TYPE
%token <svalue> IDENTIFIER
%token <intval> NUMBER
+ %token <if_repeat_stmt> IF_REPEAT

%type <expr> exp
%type <decl> declaration
@@ -248,6 +251,7 @@ statement : assign_statement SEMI { /* does nothing */ }
          | control_statement      { /* does nothing */ }
          | read_write_statement SEMI { /* does nothing */ }
          | SEMI { gen_nop_instruction(program); }
+         | if_repeat_statement SEMI
+ ;

control_statement : if_statement { /* does nothing */ }
@@ -260,6 +264,44 @@ read_write_statement : read_statement { /* does nothing */ }
                    | write_statement { /* does nothing */ }
;

+if_repeat_statement: IF_REPEAT LPAR exp RPAR
+ {

```

```

+ // Allocate the object for the label used to jump out of the statement
+ $1.l_exit = newLabel(program);
+ // Check the type of the 'if' expression
+ if ($3.expression_type == IMMEDIATE) {
+ // If the expression is constant and equal to zero, generate a jump
+ // over the rest of the code.
+ if ($3.value == 0)
+ gen_bt_instruction(program, $1.l_exit, 0);
+ } else {
+ // Generate code to check the value of the expression at runtime
+ // and jump over if it is false
+ gen_subi_instruction(program, REG_0, $3.value, 0);
+ gen_beq_instruction(program, $1.l_exit, 0);
+ }
+ // Assign the label used for the back-edge to the beginning of the loop
+ $1.l_loop = assignNewLabel(program);
+ }
+ code_block UNTIL LPAR exp RPAR
+ {
+ // Check the type of the 'until' expression
+ if ($9.expression_type == IMMEDIATE) {
+ // If the expression is constant and equal to zero, generate a jump
+ // to the beginning of the loop (the loop will be infinite!)
+ if ($9.value == 0)
+ gen_bt_instruction(program, $1.l_loop, 0);
+ } else {
+ // Generate code to check the value of the expression at runtime
+ // and jump to the beginning of the loop if it is false
+ gen_subi_instruction(program, REG_0, $9.value, 0);
+ gen_beq_instruction(program, $1.l_loop, 0);
+ }
+ // Assign the label used to exit from the statement
+ assignLabel(program, $1.l_exit);
+ }
+
+
+ assign_statement : IDENTIFIER LSQUARE exp RSQUARE ASSIGN exp
+ {
+ // Notify to 'program' that the value $6
diff --git a/acse/axe_struct.h b/acse/axe_struct.h
index 6e7b689..04643e2 100644
--- a/acse/axe_struct.h
+++ b/acse/axe_struct.h
@@ -107,6 +107,12 @@ typedef struct t_while_statement
+ * that follows the while construct */
+ } t_while_statement;

+typedef struct t_if_repeat_stmt
+{
+ t_axe_label *l_loop;
+ t_axe_label *l_exit;
+} t_if_repeat_stmt;
+
+ /* create a label */
extern t_axe_label *alloc_label(int value);

diff --git a/tests/if_repeat/if_repeat.src b/tests/if_repeat/if_repeat.src
new file mode 100644
index 0000000..4a85be9
--- /dev/null
+++ b/tests/if_repeat/if_repeat.src
@@ -0,0 +1,7 @@
+int i;
+read(i);
+if_repeat (i > 0) {
+ write(i);
+ i = i - 1;
+} until (i == 0);
+write(0);

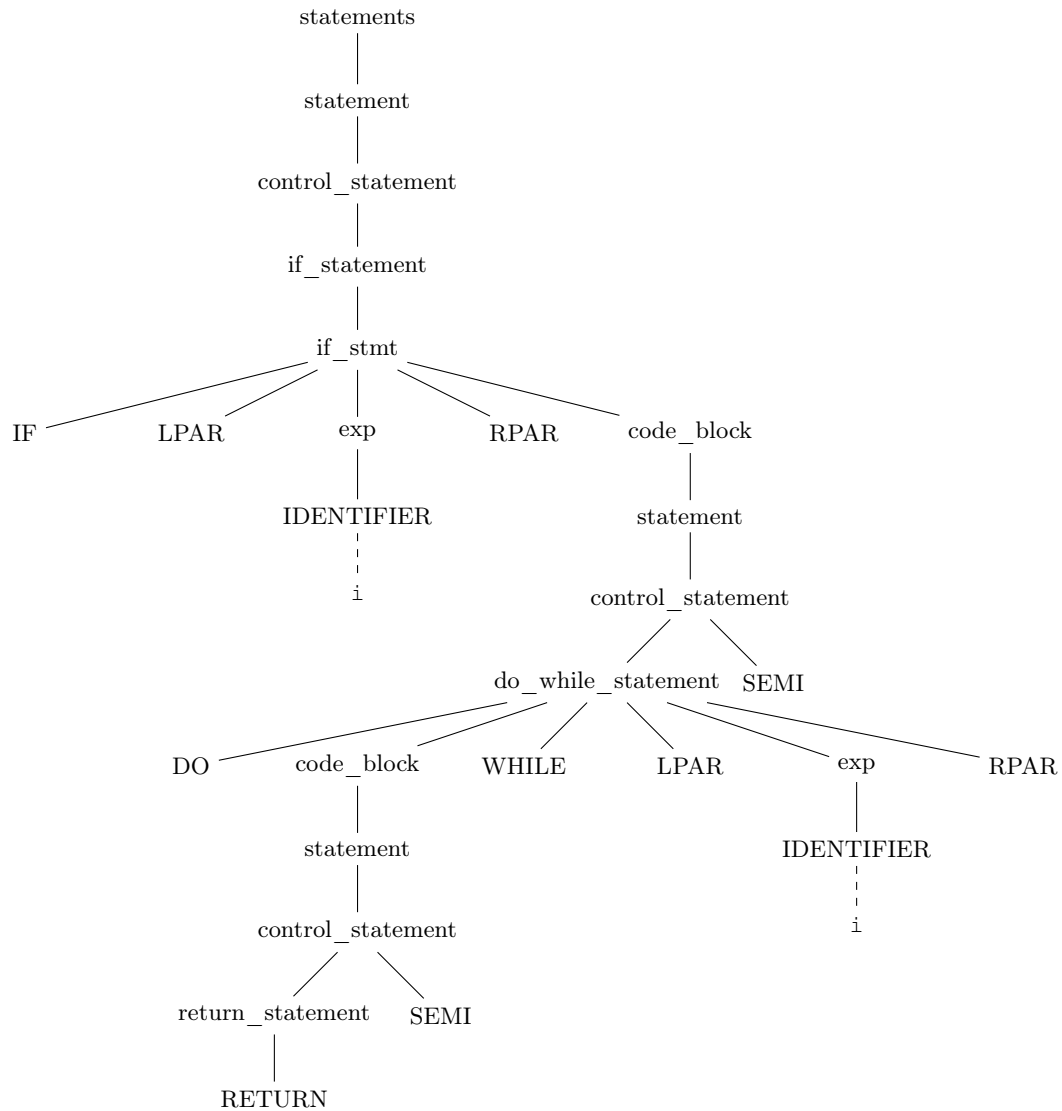
```

4. Given the following Lance code snippet:

```
if (i) do return; while (i);
```

write down the syntactic tree generated during the parsing with the Bison grammar described in Acse.y *starting from the statements nonterminal*. (5 points)

Solution



5. (**Bonus**) Write the required modifications to the token definitions and syntactic rules defined in the answers to questions (1) and (2) in order to also support a syntax that does not have the “if” part in the statement, as shown in the following example:

```
int i;  
read(i);  
repeat {  
    write(i);  
    i = i - 1;  
} until (i == 0);  
write(0);
```